

5. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100

typedef struct {
    char value[MAX_TOKEN_LENGTH];
    int quoted;
} Token;

/* Parse input and handle single quotes */
int parseInput(char *input, Token tokens[]) {
    int count = 0;
    int i = 0;

    while (input[i] != '\0' && count < MAX_TOKENS) {
        /* Skip whitespace outside quotes */
        while (input[i] == ' ' || input[i] == '\t') {
            i++;
        }

        if (input[i] == '\0')
            break;

        int j = 0;
        int quoted = 0;

        /* Handle single-quoted string */
        if (input[i] == '\'' ) {
            quoted = 1;
            i++;

            while (input[i] != '\0' && input[i] != '\0') {
                if (j < MAX_TOKEN_LENGTH - 1) {
                    tokens[count].value[j++] = input[i];
                }
                i++;
            }

            /* Missing closing quote */
            if (input[i] == '\0') {
                printf("Error: Missing closing single quote.\n");
                return -1;
            }
        }
    }
}
```

```
GNU nano 8.7.1

/* Missing closing quote */
if (input[i] == '\0') {
    printf("Error: Missing closing single quote.\n");
    return -1;
}

/* Skip closing quote */
i++;
}

/* Handle normal unquoted text */
else {
    while (input[i] != '\0' &&
           input[i] != ' ' &&
           input[i] != '\t') {
        if (input[i] == '\0') {
            printf("Error: Unexpected single quote.\n");
            return -1;
        }

        if (j < MAX_TOKEN_LENGTH - 1) {
            tokens[count].value[j++] = input[i];
        }
        i++;
    }

    tokens[count].value[j] = '\0';
    tokens[count].quoted = quoted;
    count++;
}

return count;
}

/* Display parsing result */
void displayTokens(Token tokens[], int count) {
    printf("\nParsing Result:\n");
    printf("-----\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d : %s", i + 1, tokens[i].value);

        if (tokens[i].quoted)
            printf(" [SINGLE QUOTED]");
    }
}
```

```

GNU nano 8.7.1
/* Display parsing result */
void displayTokens(Token tokens[], int count) {

    printf("\nParsing Result:\n");
    printf("-----\n");

    for (int i = 0; i < count; i++) {

        printf("Token %d : %s", i + 1, tokens[i].value);

        if (tokens[i].quoted)
            printf("    [SINGLE QUOTED]");

        else
            printf("    [NORMAL]");

        printf("\n");
    }

    printf("-----\n");
}

/* Validate parsing result */
void validateTokens(Token tokens[], int count) {

    if (count == 0) {
        printf("No tokens found.\n");
        return;
    }

    printf("Parsing result is valid.\n");
}

int main() {

    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("Single Quote Parser\n");
    printf("-----\n");

    printf("Enter input: ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    int count = parseInput(input, tokens);

    if (count == -1) {
        return 1;
    }
}

```

```

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    int count = parseInput(input, tokens);

    if (count == -1) {
        return 1;
    }

    displayTokens(tokens, count);

    validateTokens(tokens, count);

    return 0;
}

```

Result:

```
ketana_sindhu@KetaZone: ~/ x + v
ketana_sindhu@KetaZone:~/SKILL$ nano Skill5a.c
ketana_sindhu@KetaZone:~/SKILL$ gcc Skill5a.c -o s5a
ketana_sindhu@KetaZone:~/SKILL$ ./s5a
Single Quote Parser
-----
Enter input: echo 'Hello World'

Parsing Result:
-----
Token 1 : echo      [NORMAL]
Token 2 : Hello World [SINGLE QUOTED]
-----
Parsing result is valid.
ketana_sindhu@KetaZone:~/SKILL$ ./s5a
Single Quote Parser
-----
Enter input: echo '$USER'

Parsing Result:
-----
Token 1 : echo      [NORMAL]
Token 2 : $USER     [SINGLE QUOTED]
-----
Parsing result is valid.
ketana_sindhu@KetaZone:~/SKILL$ ./s5a
Single Quote Parser
-----
Enter input: echo 'Hello World from Linux'

Parsing Result:
-----
Token 1 : echo      [NORMAL]
Token 2 : Hello World from Linux [SINGLE QUOTED]
-----
Parsing result is valid.
ketana_sindhu@KetaZone:~/SKILL$ ./s5a
Single Quote Parser
-----
Enter input: echo 'Hello World
Error: Missing closing single quote.
ketana_sindhu@KetaZone:~/SKILL$ |
```

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Code:

```
GNU nano 3.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100

typedef struct {
    char value[MAX_TOKEN_LENGTH];
    int quoted;
} Token;

/* Expand a simple environment variable */
void expandVariable(char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_TOKEN_LENGTH - 1) {
        if (input[i] == '$') {
            i++;
            char variable[50];
            int k = 0;
            while (input[i] != '\0' &&
                (isalnum((unsigned char)input[i]) || input[i] == '_')) {
                if (k < 49)
                    variable[k++] = input[i];
                i++;
            }
            variable[k] = '\0';
            char *value = getenv(variable);
            if (value != NULL) {
                for (int x = 0; value[x] != '\0' &&
                    j < MAX_TOKEN_LENGTH - 1; x++) {
                    output[j++] = value[x];
                }
            }
            else {
                output[j++] = input[i++];
            }
        }
        else {
            output[j++] = input[i++];
        }
    }
    output[j] = '\0';
}

/* Parse input and handle double quotes */
int parseInput(char *input, Token tokens[]) {
    int count = 0;
    int i = 0;

    while (input[i] != '\0' && count < MAX_TOKENS) {
        /* Skip whitespace outside quotes */
        while (input[i] == ' ' || input[i] == '\t') {
            i++;
        }
        if (input[i] == '\0')
            break;
        int j = 0;
        if (input[i] == '"') {
            /* Handle double-quoted string */
            if (input[i] == '"') {
                quoted = 1;
                i++;
                while (input[i] != '\0' && input[i] != '"') {
                    if (input[i] == '$') {
                        char variablePart[MAX_TOKEN_LENGTH];
                        int k = 0;
                        while (input[i] != '\0' &&
                            (isalnum((unsigned char)input[i]) ||
                                input[i] == '_')) {
                            if (k < MAX_TOKEN_LENGTH - 1)
                                variablePart[k++] = input[i];
                            i++;
                        }
                        variablePart[k] = '\0';
                        char expanded[MAX_TOKEN_LENGTH];
                        expandVariable(variablePart, expanded);
                        for (int x = 0; expanded[x] != '\0' &&
                            j < MAX_TOKEN_LENGTH - 1; x++) {
                            tokens[count].value[j++] = expanded[x];
                        }
                    }
                    else {
                        if (j < MAX_TOKEN_LENGTH - 1)
                            tokens[count].value[j++] = input[i];
                        i++;
                    }
                }
            }
            /* Missing closing quote */
            if (input[i] == '\0') {
                printf("Error: Missing closing double quote.\n");
                return -1;
            }
            /* Skip closing quote */
            i++;
        }
        /* Handle normal unquoted token */
        else {
            while (input[i] != '\0' &&
                input[i] != ' ' &&
                input[i] != '\t') {
                if (input[i] == '"') {
                    printf("Error: Unexpected double quote.\n");
                    return -1;
                }
                if (j < MAX_TOKEN_LENGTH - 1)
                    tokens[count].value[j++] = input[i];
                i++;
            }
        }
        count++;
    }
}
```

```
GNU nano 3.7.1
int j = 0;
int quoted = 0;
/* Handle double-quoted string */
if (input[i] == '"') {
    quoted = 1;
    i++;
    while (input[i] != '\0' && input[i] != '"') {
        if (input[i] == '$') {
            char variablePart[MAX_TOKEN_LENGTH];
            int k = 0;
            while (input[i] != '\0' &&
                (isalnum((unsigned char)input[i]) ||
                    input[i] == '_')) {
                if (k < MAX_TOKEN_LENGTH - 1)
                    variablePart[k++] = input[i];
                i++;
            }
            variablePart[k] = '\0';
            char expanded[MAX_TOKEN_LENGTH];
            expandVariable(variablePart, expanded);
            for (int x = 0; expanded[x] != '\0' &&
                j < MAX_TOKEN_LENGTH - 1; x++) {
                tokens[count].value[j++] = expanded[x];
            }
        }
        else {
            if (j < MAX_TOKEN_LENGTH - 1)
                tokens[count].value[j++] = input[i];
            i++;
        }
    }
    /* Missing closing quote */
    if (input[i] == '\0') {
        printf("Error: Missing closing double quote.\n");
        return -1;
    }
    /* Skip closing quote */
    i++;
}
/* Handle normal unquoted token */
else {
    while (input[i] != '\0' &&
        input[i] != ' ' &&
        input[i] != '\t') {
        if (input[i] == '"') {
            printf("Error: Unexpected double quote.\n");
            return -1;
        }
        if (j < MAX_TOKEN_LENGTH - 1)
            tokens[count].value[j++] = input[i];
        i++;
    }
    count++;
}
```

```

GNU nano 8.7.1
    }
    if (j < MAX_TOKEN_LENGTH - 1)
        tokens[count].value[j++] = input[i];
        i++;
    }
    tokens[count].value[j] = '\0';
    tokens[count].quoted = quoted;
    count++;
}
return count;
}
/* Display parsing results */
void displayTokens(Token tokens[], int count) {
    printf("\nParsing Result:\n");
    printf("-----\n");
    for (int i = 0; i < count; i++) {
        printf("Token %d : %-25s ", i + 1,
            tokens[i].value);
        if (tokens[i].quoted)
            printf("[DOUBLE QUOTED]\n");
        else
            printf("[NORMAL]\n");
    }
    printf("-----\n");
}
/* Validate output */
void validateOutput(Token tokens[], int count) {
    if (count == 0) {
        printf("No tokens found.\n");
        return;
    }
    printf("Parsing output is valid.\n");
}
int main() {
    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];
    printf("Double Quote Parser\n");
    printf("-----\n");
    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);
    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';
    int count = parseInput(input, tokens);
    if (count == -1)
        return 1;
    displayTokens(tokens, count);
    validateOutput(tokens, count);
    return 0;
}

```

Result:

```

ketana_sindhu@KetaZone: ~/ × + ∨
ketana_sindhu@KetaZone:~/SKILL$ nano Skill5b.c
ketana_sindhu@KetaZone:~/SKILL$ gcc Skill5b.c -o s5b
ketana_sindhu@KetaZone:~/SKILL$ ./s5b
Double Quote Parser
-----
Enter command: echo "Hello World"

Parsing Result:
-----
Token 1 : echo [NORMAL]
Token 2 : Hello World [DOUBLE QUOTED]
-----
Parsing output is valid.
ketana_sindhu@KetaZone:~/SKILL$ ./s5b
Double Quote Parser
-----
Enter command: echo "Hello World from Linux"

Parsing Result:
-----
Token 1 : echo [NORMAL]
Token 2 : Hello World from Linux [DOUBLE QUOTED]
-----
Parsing output is valid.
ketana_sindhu@KetaZone:~/SKILL$ export USER=Ketana
ketana_sindhu@KetaZone:~/SKILL$ echo "Hello $USER"
Hello Ketana
ketana_sindhu@KetaZone:~/SKILL$ ./s5b
Double Quote Parser
-----
Enter command: echo "Hello $USER"

Parsing Result:
-----
Token 1 : echo [NORMAL]
Token 2 : Hello Ketana [DOUBLE QUOTED]
-----
Parsing output is valid.
ketana_sindhu@KetaZone:~/SKILL$ |

```